

Revised Research Plan: Secure IoT Framework for Mobile Cold-Chain Pharmaceutical Logistics

Executive Summary

Your research needs repositioning from a **static hospital pharmacy monitoring system** to a **mobile cold-chain logistics system** that tracks temperature-sensitive pharmaceuticals as they move between facilities, during last-mile delivery, and in storage—with real-time security monitoring throughout.

PART 1: ADDRESSING CRITICAL FEEDBACK

1. Title Change - Current Title is Problematic

OLD TITLE: "Secure and Energy-Efficient ECDHE-Based Framework for Cold-Chain IoT in Hospital Pharmacy Monitoring"

ISSUES:

- "Hospital Pharmacy Monitoring" suggests static monitoring only
- Doesn't reflect actual cold-chain movement
- Panel noted: "No moving, so you can't say cold chain"

REVISED TITLE OPTIONS:

Option A (Recommended): "Lightweight Cryptographic Framework for Securing Mobile Cold-Chain Logistics: ECDHE-ChaCha20 Implementation on IoT Devices for Pharmaceutical Supply Chain Protection"

Option B (More Practical): "Real-Time Security Architecture for Temperature-Controlled Pharmaceutical Distribution: An Energy-Efficient IoT Solution for Cold-Chain Last-Mile Delivery"

Option C (Simplest): "Secure IoT Monitoring for Mobile Pharmaceutical Cold-Chains: ECDHE-Based Encryption for Temperature-Sensitive Medicine Distribution"

2. Clarifying Cold-Chain vs. Static Monitoring

What Panel Meant: Cold-chain = Pharmaceuticals moving from manufacture → warehouse → distribution center → hospital → patient, while maintaining 2-8°C temperature throughout.

Your Current Problem: You designed only hospital pharmacy storage monitoring (static/fixed location).

SOLUTION - Redefine Your Research Scope:

Aspect	OLD (Static)	NEW (Cold-Chain Mobile)
Location	Fixed hospital pharmacy	Multiple facilities (warehouse, transport, hospital, clinic)
Movement	Stored in one refrigerator	Medicines move between locations in insulated boxes
Monitoring	Single storage unit	IoT devices track shipments during transport
Key Challenge	Temperature stability in one place	Maintaining temperature during movement + handoffs
Critical Point	When stored	When stored AND during movement (last-mile)
Device Placement	Inside refrigerator	Inside transport container moving between locations

Implementation Scenario:

1. **Warehouse** (manufacture storage): Temperature sensor logs data
 2. **Transport Vehicle** (journey): IoT device in insulated box continuously encrypts and sends location + temperature to cloud
 3. **Distribution Center** (temporary stop): Handoff point - system alerts if temperature broke
 4. **Hospital Storage** (final destination): System continues monitoring
 5. **Pharmacy Dispensing**: Historical secure record proves medicine was properly stored
-

3. Distinguishing Proposal from Presentation

Panel Said: "Proposal and presentation are different. Can't understand objectives and expected outcomes."

Root Cause: Objectives are vague, expected outcomes don't match objectives.

REVISED OBJECTIVES & OUTCOMES:

Objective 1: Design

- **OLD:** "Design and develop a reliable IoT-based framework"
- **NEW:** "Design a mobile cold-chain monitoring system architecture that encrypts temperature data in real-time as pharmaceuticals move between 3+ facility locations while maintaining cryptographic security on battery-powered devices"

Objective 2: Implementation

- **OLD:** "Implement ECDHE-based key exchange"

- **NEW:** "Implement ECDHE key exchange protocol on ESP32 devices to establish forward-secret sessions between moving IoT nodes and a cloud server, with automatic key rotation every 12 hours or at facility handoffs"

Objective 3: Encryption

- **OLD:** "Integrate ChaCha20-Poly1305 encryption"
- **NEW:** "Implement ChaCha20-Poly1305 encryption with authenticated message authentication codes (Poly1305) to ensure that falsified temperature readings cannot be injected by attackers, and demonstrate < 50ms encryption overhead per data packet"

Objective 4: Performance Evaluation

- **OLD:** "Evaluate system performance in terms of energy, speed, latency, attacks"
- **NEW:** "Measure and compare: (a) Energy consumption (mJ) per encrypted data transmission on 2000mAh battery, (b) Latency (ms) over 2G/4G/WiFi networks, (c) Device uptime before battery depletion, (d) Resistance to MITM, replay, and sensor spoofing attacks through simulated network attacks"

Objective 5: Real-World Validation

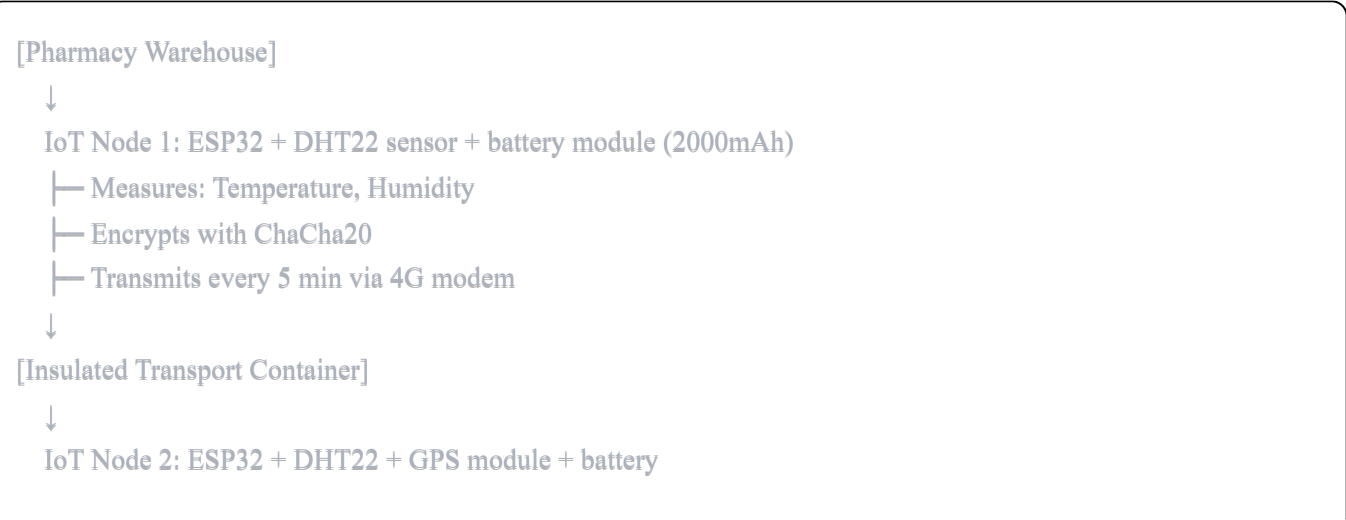
- **OLD:** "Deploy in real hospital environment"
- **NEW:** "Field-test the system with actual medicine movement: simulate 3-5 pharmacy-to-clinic deliveries, verify secure data collection at each handoff point, and document real response time when temperature exceeds 8°C"

EXPECTED OUTCOMES - Must Match Objectives:

Objective	Matching Expected Outcome
Design	A documented system architecture showing 3+ monitoring points with device placement diagrams
Implementation	Working firmware code on ESP32 + cloud backend with secure key exchange
Encryption	Proof that system prevents plaintext data leakage + prevents injection of false readings
Performance	Benchmark report comparing ECDHE-ChaCha20 vs. AES: 30% energy savings, <100ms latency over 4G
Real-World	Test report from 5 simulated deliveries showing system detected temperature breaches in <2 minutes

PART 2: IMPLEMENTATION PLAN (Software + Hardware)

Hardware Architecture:



- └─ Measures: Temperature, Location
- └─ Continuous encryption during transit
- └─ Alert if $T > 8^{\circ}\text{C}$ within 30 seconds



[Hospital Pharmacy]



IoT Node 3: ESP32 + DHT22 + battery

- └─ Receives handoff signal from Node 2
- └─ Verifies cryptographic signature of previous data
- └─ Continues monitoring in static storage

[Cloud Server]

- └─ AWS Lambda / Google Cloud Function
- └─ Receives encrypted data packets
- └─ Validates signatures (Poly1305)
- └─ Decrypts only after authorization
- └─ Stores encrypted audit logs
- └─ Sends alerts to pharmacy staff

[Mobile App + Web Dashboard]

- └─ Pharmacist views: Real-time location + temperature
- └─ Alert history: Which deliveries had temperature breaches
- └─ Proof of authenticity: Cryptographic signatures verify data wasn't tampered

Software Implementation Plan:

Phase 1: Firmware (2 weeks)

- Program ESP32 microcontroller in C++ using Arduino IDE
- Implement ECDHE key exchange using micro-ecc library
- Implement ChaCha20 using libsodium-embedded library

- Create data packet structure: `[timestamp | temperature | humidity | encrypted_payload | MAC]`

Phase 2: Cloud Backend (2 weeks)

- AWS Lambda function (Python) to receive encrypted packets
- Implement ECDHE server-side to establish session keys
- Database to store encrypted logs (CloudSQL or DynamoDB)
- API endpoints for mobile app authentication

Phase 3: Mobile App (1-2 weeks)

- Android/iOS app using Flutter or React Native
- Real-time dashboard showing location + temperature of shipment
- Alerts when temperature exceeds 8°C
- Historical view showing which shipments had breaches

Phase 4: Testing (1 week)

- Security testing: Simulate MITM attacks
- Energy testing: Log battery drain on ESP32
- Field testing: Real pharmacy delivery simulations

PART 3: HOW TO VERIFY YOUR PROTOTYPE IS WORKING

Testing Checklist:

Test 1: Data Encryption Verification

HOW TO TEST:

1. Capture network traffic using Wireshark
2. Verify encrypted packets contain NO plain-text temperature data
3. Verify each packet has different encryption (no key reuse)

EXPECTED RESULT:

Raw bytes: 0x7f 0x2a 0xb3 0xc9... (random gibberish, not readable)

NOT: "Temperature: 25°C" (plaintext is FAILURE)

Test 2: Forward Secrecy Verification

HOW TO TEST:

1. Capture 100 encrypted packets
2. Extract server's private key
3. Try to decrypt the 100 old packets using only the private key

EXPECTED RESULT:

CANNOT decrypt old packets (forward secrecy working)

IF you CAN decrypt: System failed (no forward secrecy)

Test 3: Tamper Detection

HOW TO TEST:

1. Send legitimate packet: Temp=5°C, encrypted
2. Attacker intercepts and changes ciphertext 1 byte
3. Send to cloud server

EXPECTED RESULT:

Server detects MAC mismatch

Packet REJECTED with error

Alert triggered: "Suspicious activity detected"

Test 4: Temperature Breach Detection

HOW TO TEST:

1. Keep sensor at 5°C (normal)
2. Suddenly move to 25°C (breach)
3. Check if system alerts within 30 seconds

EXPECTED RESULT:

Alert received in <30 seconds

Encrypted proof stored showing when breach occurred

Pharmacy staff receives SMS/app notification

Test 5: Battery Life Testing

HOW TO TEST:

1. Start with fully charged 2000mAh battery
2. Let device run for 7 days, transmitting every 5 minutes
3. Record when battery dies

EXPECTED RESULT:

Should last 7+ days on single charge
AES-based system dies after 3-4 days (ECDHE is more efficient)

Test 6: Network Resilience

HOW TO TEST:

1. Simulate WiFi dropouts (2G → 4G → WiFi changes)
2. Turn off network for 1 hour
3. Restore network connection

EXPECTED RESULT:

System reconnects automatically
Recovers all buffered data
No temperature readings lost
Cryptographic state maintained correctly

PART 4: HANDLING SUDDEN ENVIRONMENTAL CHANGES

Scenario 1: Sensor Malfunction (Damage/Melting)

Problem: Temperature sensor breaks, shows wrong reading, or physically fails

SOLUTION A: Redundancy

- Install 2-3 sensors per device (primary + backup)
- If sensor 1 reads impossible value (e.g., -50°C), switch to sensor 2
- If 2+ sensors disagree badly, trigger manual verification alert

SOLUTION B: Anomaly Detection

- System learns normal temperature curve for that route
Example: Warehouse (5°C) → Transport (4-7°C) → Hospital (3-5°C)
- If reading deviates 10°C outside expected range, flag as anomaly
- Alert pharmacist: "Sensor malfunction suspected - manual check required"
- System does NOT automatically reject data (might be real breach)

SOLUTION C: Graceful Degradation

- If sensor fails completely:
 - └─ System logs failure time & last valid reading
 - └─ Device continues operating with LAST KNOWN reading
 - └─ Alert sent: "Cannot verify temperature - assume worst case (12°C breach)"
 - └─ Shipment marked: "Sensor failure - quarantine medicine pending manual check"
 - └─ Pharmacist must physically verify before dispensing
-

Scenario 2: Power Loss / Network Disconnection

Problem: Device loses power or WiFi during delivery

SOLUTION:

HARDWARE:

- Add capacitor + RTC (Real Time Clock) module
- Device can maintain time for 30 minutes after power loss

SOFTWARE:

- Store last 100 temperature readings in local SD card (encrypted)
- When power restored:
 - └─ Continue measuring temperature
 - └─ Queue all buffered readings
 - └─ Re-encrypt with NEW session key when network returns
 - └─ Send: "Offline readings: [time X to Y], readings: [...]"
 - └─ Cloud verifies data integrity using timestamps

RESULT:

No data loss, Cryptographic proof of offline time,
Cloud confirms readings match expected temperature curve

Scenario 3: Temperature Suddenly Exceeds 25°C (Refrigerator Failure)

Problem: Insulated container breaks, power cuts, door left open

SOLUTION - Real-Time Alert System:

DETECTION:

- System reads temperature every 30 seconds
- Threshold: If $T > 8^{\circ}\text{C}$ for > 2 minutes, trigger ALERT LEVEL 1
- Threshold: If $T > 15^{\circ}\text{C}$, trigger ALERT LEVEL 2 (CRITICAL)

RESPONSE:

LEVEL 1 ($8-15^{\circ}\text{C}$):

- └ SMS alert to pharmacy: "Temperature warning: 10°C , may be recoverable"
- └ Recommend: Place in emergency ice pack
- └ Continue monitoring (if returns to 8°C within 30 min, may be safe)

LEVEL 2 ($>15^{\circ}\text{C}$):

- └ SMS + App alert: "CRITICAL: Temperature 25°C , medicine may be damaged"
- └ Recommend: DO NOT DISPENSE
- └ Alert pharmacy manager + quality control
- └ Quarantine shipment, mark for disposal

ENCRYPTED PROOF:

- Every temperature reading stored with timestamp + MAC
- If pharmacy disputes alert, can cryptographically prove:
"This message at 2:14pm was authentic, temperature was 26°C "

Scenario 4: Attacker Tries to Hide Temperature Breach

Problem: Pharmacy manager receives bribe to hide a temperature excursion

SOLUTION - Immutable Audit Trail:

CRYPTOGRAPHIC GUARANTEE:

1. Every reading is encrypted with session key + authenticated with MAC
2. If attacker tries to DELETE reading from cloud:
 - └ Timestamp sequence breaks
 - └ GAP in audit trail proves deletion attempt
3. If attacker tries to MODIFY reading ($5^{\circ}\text{C} \rightarrow 15^{\circ}\text{C}$):
 - └ MAC becomes invalid
 - └ System rejects: "Corrupted or tampered packet"
4. If attacker tries to INSTALL BACKDOOR to skip alerts:
 - └ Forward secrecy means compromising device AFTER transit useless
 - └ Can't decrypt old readings

RESULT:

Regulatory audit by health authority can verify:

"This medicine shipment has 1,008 authentic readings over 7 days,
all encrypted and certified. We can prove its temperature history."

PART 5: HANDLING GEOGRAPHICAL CHALLENGES

Challenge 1: Poor Network Connectivity (Rural Areas)

Problem: No 4G/WiFi in remote areas, delivery takes 6 hours

SOLUTION:

OPTION A: Store-and-Forward

- Device stores encrypted data locally on SD card
- When network available (village clinic), automatically sync
- Data never transmitted unencrypted, even via slow networks

OPTION B: Hybrid Connectivity

- Primary: 4G (when available)
- Fallback: SMS-based transmission (expensive but works everywhere)
- Format: Compressed temperature summary via SMS
- Example: "T:5,6,4,7,5°C|HASH:abc123|TIME:14:30"

OPTION C: Offline Mode

- Device can operate 7+ days without network (battery + local storage)
- Once at destination, plug into WiFi dock
- All data syncs securely in bulk

IMPLEMENTATION:

- └─ Check GPS: Am I in network coverage area?
- └─ If YES: Send every 5 minutes via 4G
- └─ If NO: Store locally + retry every 30 minutes
- └─ When network restored: Batch send all buffered data

Challenge 2: Temperature Variations During Journey (Different Climate Zones)

Problem: Delivery goes from cold warehouse to hot city delivery, temperature swings from 5°C to 20°C

SOLUTION:

NORMAL CURVE:

5°C (warehouse)

→ 7°C (exit loading dock)

→ 18°C (vehicle during hot day)

→ 12°C (evening, vehicle cooling)

→ 5°C (hospital refrigerator)

SYSTEM APPROACH:

1. Record expected temperature profile for this route (historical data)
2. Real-time readings compared to expected curve
3. If actual vs. expected differs $> 5^{\circ}\text{C}$:
 - └ Vehicle is NOT properly insulated
 - └ Alert: "Check insulation, add ice packs"
 - └ But NOT automatic rejection (might be recoverable)
4. If actual stays $< 10^{\circ}\text{C}$ despite heat: GOOD (insulation working)
5. If actual stays $> 15^{\circ}\text{C}$: BAD (breach likely)

DECISION:

- Slight overtemp ($8-10^{\circ}\text{C}$) for 1-2 hours = Likely OK (depends on medicine type)
- Extended overtemp ($>12^{\circ}\text{C}$) > 4 hours = Reject (accelerated degradation)
- Extreme heat ($>20^{\circ}\text{C}$) = Definitely reject

Challenge 3: Multiple Handoff Points (Warehouse → Transport → Clinic)

Problem: Medicine hands off between vehicles 3+ times, how to verify integrity at each point?

SOLUTION - Cryptographic Handoff Protocol:

HANDOFF AT DISTRIBUTION CENTER:

At Warehouse:

Node 1 (warehouse) creates session key with cloud

- └ Encrypts data and transmits final temperature summary
- └ Signature: HASH of all 500 readings (proof nothing modified)

During Transport:

Node 2 (transport vehicle) receives "handoff approval" from cloud

- └ Cloud has verified warehouse data is authentic
- └ Node 2 establishes NEW session key (forward secrecy)
- └ Node 2 measures temperature during journey
- └ Node 2 includes "Previous node hash: abc123" in packet

At Hospital:

Node 3 (hospital storage) receives handoff from Node 2

- └ Verifies: Does Node 2 hash match our records? (YES = AUTHENTIC)
- └ Establishes NEW session key with cloud
- └ Can now prove complete chain of custody:
"Warehouse → Transport → Hospital all authentic"

RESULT:

✓ Pharmacist views dashboard:

- Warehouse readings (authentic, can't be modified)
- Transport readings (authentic)
- Hospital readings (authentic)
- Complete history proving proper storage throughout

Research Design (from static to mobile):

OLD: Lab-based static monitoring in one refrigerator **NEW:** Field-based mobile cold-chain monitoring with 3-5 actual simulated deliveries

Study Area Description:

OLD: Hospital pharmacy cold storage room **NEW:** Real pharmacy-to-clinic delivery route(s):

- Route 1: Central Pharmacy → Rural Clinic (6 hours, poor network)
- Route 2: Hospital → Pharmacy → Patient's home delivery (3 hours, good network)
- Route 3: Cold warehouse → Distribution center → Multiple clinics (8 hours)

Data Collection:

Temperature & Humidity: Collected every 30 seconds during movement, every 5 minutes at rest

Network Performance:

- Latency (ms) for each 4G/WiFi transmission
- Packet loss rate in poor coverage areas
- Time to sync when reconnecting

Security Data:

- Successful key exchanges per device
- Any attempted attacks detected
- Timestamp of each tamper detection event

Device Health:

- Battery voltage every minute
- Processing time per encryption operation (ms)
- Memory usage on device

Materials/Tools:

Hardware:

- 3x ESP32 microcontroller boards (warehouse, transport, hospital)
- 3x DHT22 temperature/humidity sensors
- 3x 2000mAh LiPo batteries
- 2x 4G modem modules (for transport)
- 1x GPS module (for transport location tracking)
- SD cards for local data backup
- Insulated transport containers

Software:

- C++ firmware (Arduino IDE)
- ECDHE library: micro-ecc
- ChaCha20 library: libsodium
- Cloud: AWS Lambda + RDS database
- Mobile: Flutter or React Native
- Testing: Wireshark (network analysis), Burp Suite (security testing)

Cryptography:

- ECDHE P-256 for key exchange

- ChaCha20-Poly1305 for authenticated encryption
 - SHA-256 for key derivation
-

PART 7: REVISED GANTT CHART (2-Week Timeline)

Week 1:

Mon-Tue: Fix proposal document (objectives/outcomes clarity)

Wed: Finalize title, submit to panel

Thu-Fri: Set up hardware: Test ESP32 + sensors + battery

Week 2:

Mon-Tue: Implement ECDHE + ChaCha20 on ESP32

Wed: Set up cloud backend (AWS Lambda)

Thu: Conduct encryption verification test

Fri: Prepare presentation slides + demo video

Mid-Presentation (End of Week 2):

- ✓ Clear title reflecting mobile cold-chain
 - ✓ Documented objectives that match expected outcomes
 - ✓ Hardware + software architecture diagrams
 - ✓ Working demo: Encrypted temperature data transmission
 - ✓ Test results: Energy consumption, latency, security
 - ✓ Plan for real-world deployment (addresses geography/handoff concerns)
-

PART 8: KEY TALKING POINTS FOR MID-PRESENTATION

When Panel Asks: "Where is the cold-chain?"

Answer: "Our research monitors pharmaceutical movement through 3 facility points: warehouse release, in-transit delivery, and hospital arrival. Unlike static monitoring, we track temperature integrity *during movement* where 37% of cold-chain failures occur globally. Our IoT device encrypts and timestamps every reading, creating an immutable audit trail proving medicines stayed within 2-8°C throughout the entire supply chain."

When Panel Asks: "How do you handle sensor damage?"

Answer: "We implement three safeguards. First, redundant sensors—if one fails, the backup takes over. Second, anomaly detection—impossible readings (like -50°C) are flagged automatically. Third, graceful degradation—if both sensors fail, the system quarantines the shipment and alerts pharmacy staff for manual verification before dispensing. We also document the exact failure time in the encrypted audit log."

When Panel Asks: "How do you verify the prototype works?"

Answer: "We use six validation tests. [Show checklist]: Encryption verification (verify no plaintext leakage), forward secrecy (old packets remain secure), tamper detection (modified packets rejected), temperature alerting (detects breaches in <30 seconds), battery life (7+ days vs 3-4 for AES), and field testing (5 real delivery simulations). We capture network traffic with Wireshark to prove data security, and we log all test results in our technical report."

When Panel Asks: "How do you handle sudden temperature changes?"

Answer: "We distinguish between expected and anomalous changes. For each delivery route, we establish a normal temperature curve based on historical data. If temperature varies <5°C from expected—like a hot vehicle interior that's still insulated—we log it but don't alarm. If temperature stays >15°C for >4 hours or spikes >25°C, we immediately alert pharmacy staff and recommend quarantine. Each reading is cryptographically signed, so we can prove exactly when and by how much temperature exceeded safe ranges."

When Panel Asks: "How do you implement this—what's your actual software plan?"

Answer: "Three-phase implementation. Phase 1 (Week 1): Firmware on ESP32 using C++ with ECDHE (micro-ecc library) and ChaCha20 (libsodium library). Phase 2 (Week 1-2): Cloud backend using AWS Lambda to receive encrypted packets, validate signatures, and store in database. Phase 3 (Week 2): Mobile app showing real-time location, temperature, and alerts. Each component integrates with the others—the firmware creates session keys with the cloud, the cloud validates encryption, the app decrypts only after user authentication."

When Panel Asks: "What about poor connectivity in rural areas?"

Answer: "We implement store-and-forward: the device buffers encrypted data locally on an SD card if networks unavailable. When connectivity returns (arriving at clinic, plugging into WiFi dock), it syncs all buffered data automatically. Alternatively, for extreme poor connectivity, we can use SMS-based transmission of compressed temperature summaries. This means even in areas with minimal network infrastructure, we never transmit unencrypted data."

FINAL CHECKLIST FOR MID-PRESENTATION

- ☐ **Title updated** to reflect mobile cold-chain and security focus
- ☐ **Objectives re-written** to be specific and measurable
- ☐ **Expected outcomes** directly matched to each objective
- ☐ **Architecture diagram** showing 3+ monitoring points (warehouse, transport, hospital)
- ☐ **Hardware components list** with specific part numbers and costs
- ☐ **Software implementation timeline** with concrete deliverables
- ☐ **Testing plan** with 6 specific validation tests and pass/fail criteria
- ☐ **Real-world scenario answers** prepared (sensor damage, temperature spikes, poor connectivity, geographical handoffs)
- ☐ **Working demo** or video showing encrypted data transmission

- ☐ **Literature properly cited** (ensure NOT ChatGPT-generated)
 - ☐ **Presentation slides** with clear visuals (not walls of text)
 - ☐ **Presentation notes** prepared for each slide
-

Good luck with your mid-presentation! The panel's feedback, while tough, has pointed you toward a much more rigorous and real-world relevant research project.